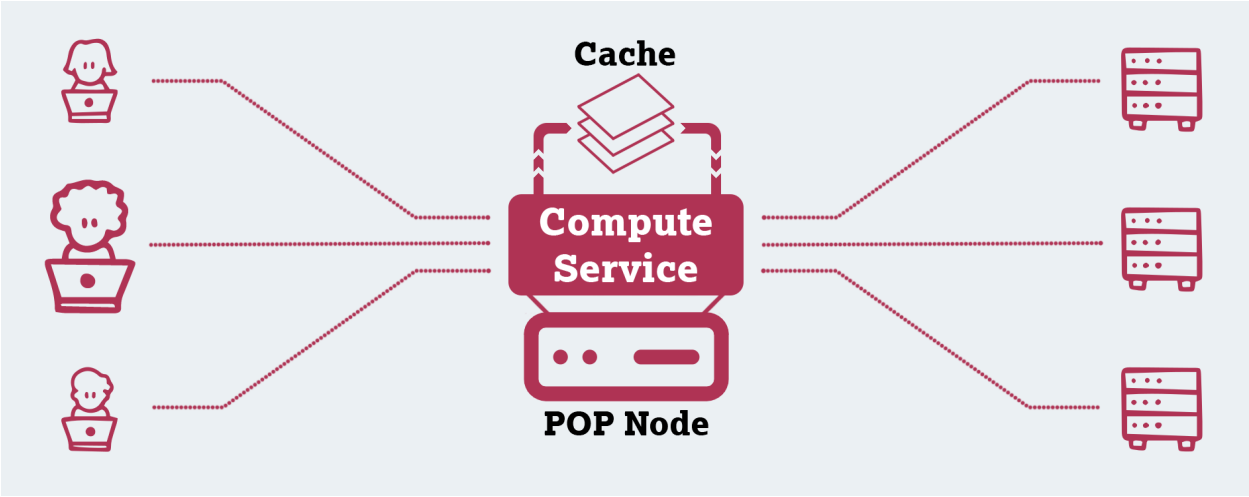
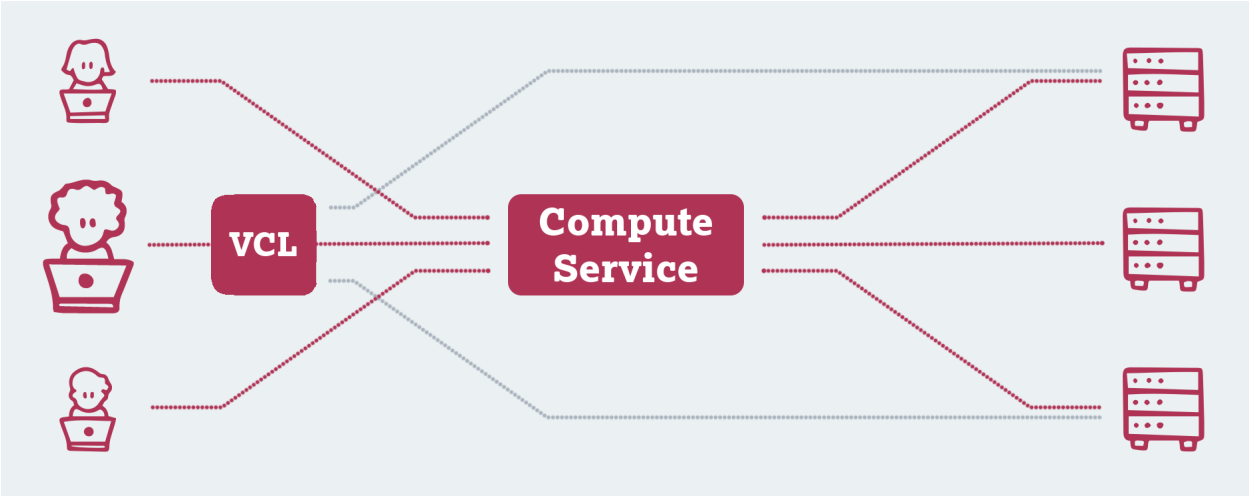
103: Compute Features and Functionalities

Lesson 1: Service Chaining

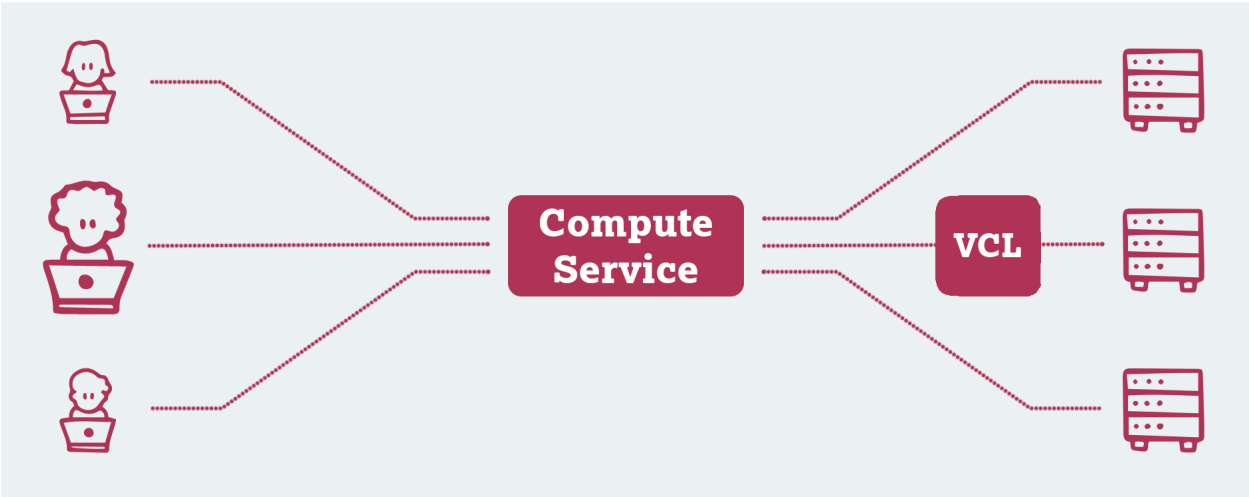


Let's outline and identify the parts of our default Compute service diagram so we're clear on the visual language. On the left, we have end-users sending requests to the Compute Service. Some diagrams include multiple end-users when relevant or just the one middle end-user. The center block is the Compute service. It is supported and runs on a POP Node, and includes the planned caching functionality. To the right of the service are the various backends/origins that content would be requested from.



As we will see with some of our diagrams, other systems may exist between either the end-users on the left and/or the backend/origins on the right.The first example of this would be linking a Deliver (VCL) service in front of the Compute service. At Fastly we call this **Service Chaining**. The VCL service can request and cache responses from the Compute service, minimizing requests and thus expenses on the Compute service. This chaining does not limit the two services to only interact with each other. The Compute service still operates like a normal service and can still receive requests directly. The VCL service, or any service chaining

in front of another, simply has one of its many possible backends configured to use another service.

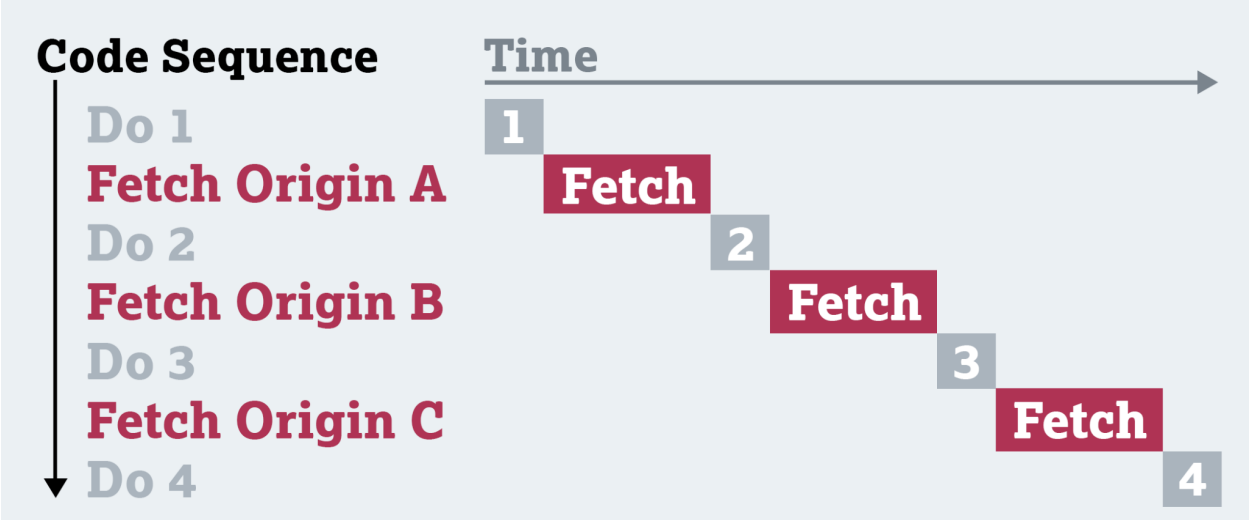


If you can Service Chain a VCL service in front of Compute, then you can chain it behind as well. Now the Compute Service is leaning on the cached content in the VCL service, allowing it to more quickly gather assets and responses on the Edge. As Fastly builds more features into Compute's native caching functionality, the VCL service becomes somewhat redundant.

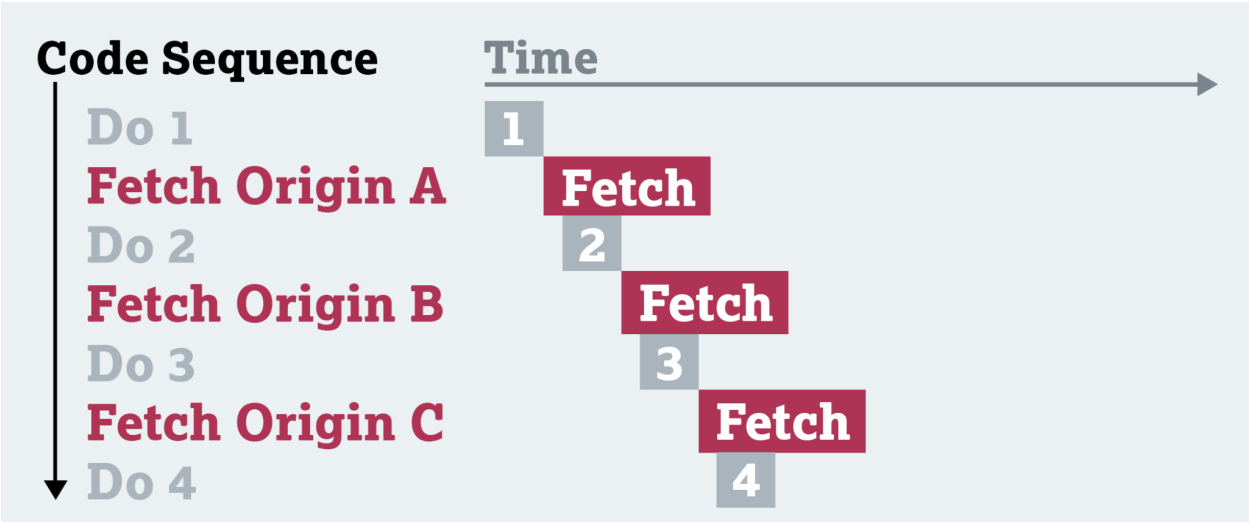
A common use case for Compute in front of VCL is request routing. Compute already has better capabilities than VCL in this area, and Fastly has released an open-source C@E JavaScript library called expressly and accompanying code examples to make it easy for customers to adopt. Fastly's intent is to let users focus on their business logic rather than on implementing low level code themselves.

Relevant Links:
https://developer.fastly.com/learning/concepts/service-chaining/

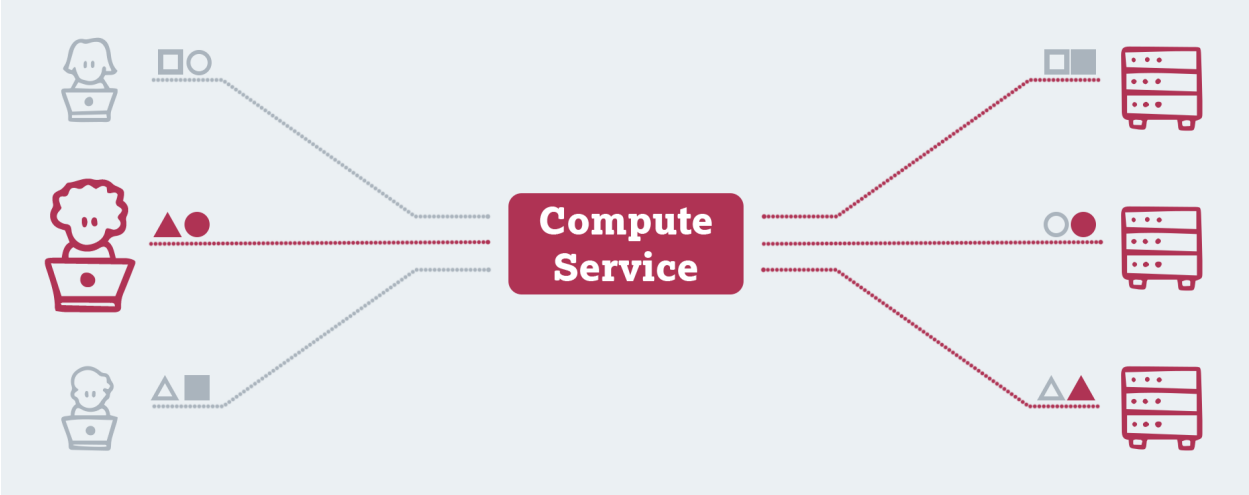
Lesson 2: Compute Functionality



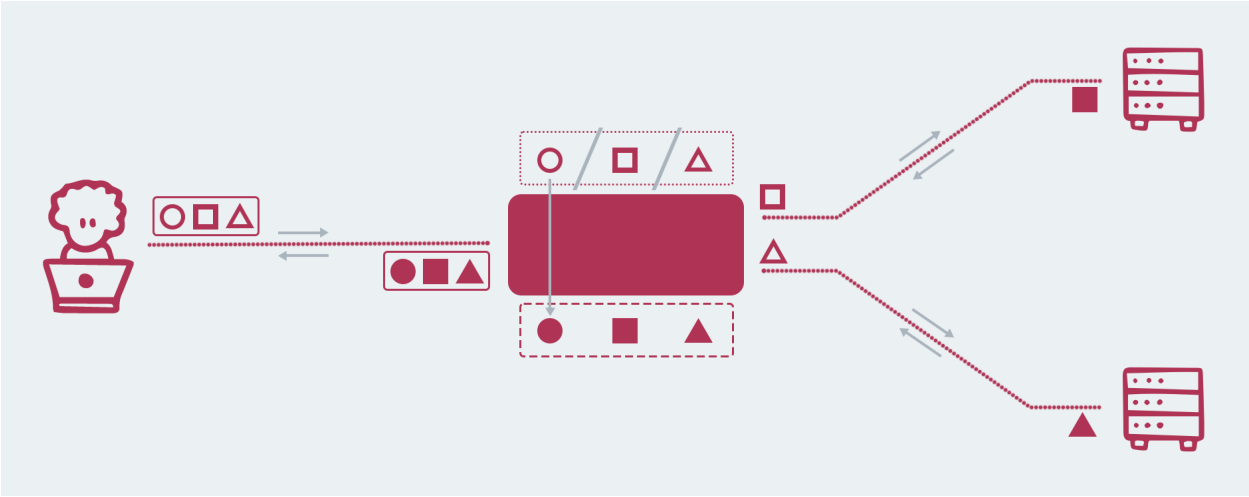
A major ability of Compute that differentiates it from our Deliver offering (VCL) is not just that it allows for multiple origin requests per client request, but that within the execution of the code those origin requests are **Asynchronous**. Let's break down what that means. Generally, code executes or runs line by line and function by function as in the abstracted image above. The first line, "Do 1", is a normal line of code that takes a short amount of time to complete. The next line, "Fetch Origin A", is a function which might contain multiple lines or steps, and takes longer to complete. For all the lines and functions in this example, note that each one only starts when the prior one has completed. So "Do 2" must wait until the "Fetch Origin A" function completes and returns with a result, usually in the form of a Response from the origin. That can lead to a ton of wasted time waiting for a response when the rest of the code could be running. Additionally, connecting to a new origin in VCL requires a restart, and the number of restarts is capped in order to prevent loops. The solution to this is to allow functions to run Asynchronously.



Many programming languages support Asynchronous execution of code, allowing two or more processes to run in parallel, i.e. at the same time. Each of the programs supported by Compute includes this functionality. Compute's origin requests run Asynchronously, allowing the code that follows after it to start and continue executing immediately. "Do 2" doesn't have to wait for "Fetch Origin A" to complete. Combined with Multi-Request, Asynchronous Multi-Request allows the Compute Instance to perform similar to a browser and start the requests for needed content as soon as the program's logic decides it is needed.

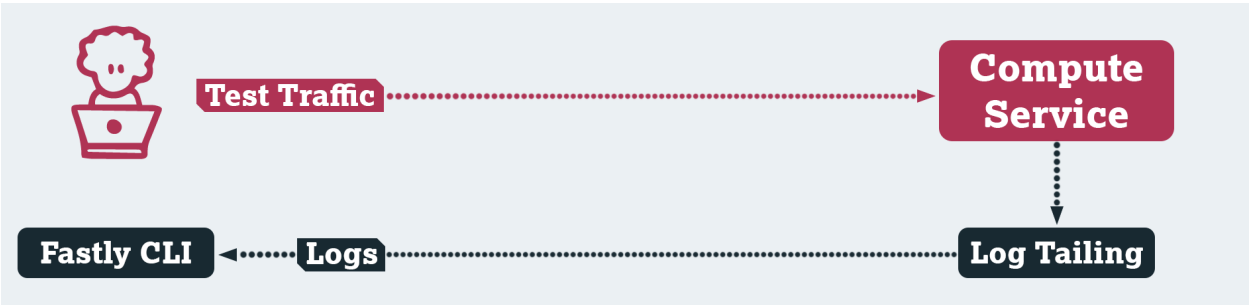


Leaning on Compute's **Asynchronous Multi-Request** functionality, the Compute service can request multiple assets from different origins and stitch those assets together into one response to the end-user. This allows for a highly customized experience for users on the Edge. HTML pages, videos with targeted ads, query and API manipulation both up and downstream, are just a few examples of what can be modified based on user data. This is possible because Compute services are able to modify all parts of the HTTP Request and/or the Response, including **Modifying the Body Object**, not just the HTTP Headers. This is how a requested HTML file, the response body object, can be stitched together via assets from multiple origins.



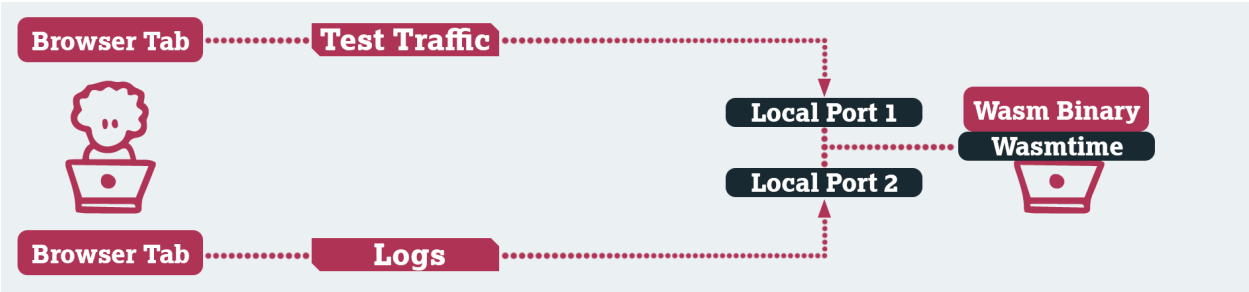
Queries and API calls can also be manipulated. They can be **Unbundled** and **Rebundled** on a Compute service as another form of content stitching. **Bundling** is when multiple queries or API calls are grouped together, i.e. bundled, into one request or response. In our example above, the end-user sends a bundle of three queries. The service logic can then Unbundle them, or pull them apart, into separate pieces of data. The circle is handled directly by the service, while the square and triangle need to be sent to their separate origins for a response. Once the service has all three query responses ready, they can be Rebundled together and sent back to the end-user.

Lesson 3: Compute Features

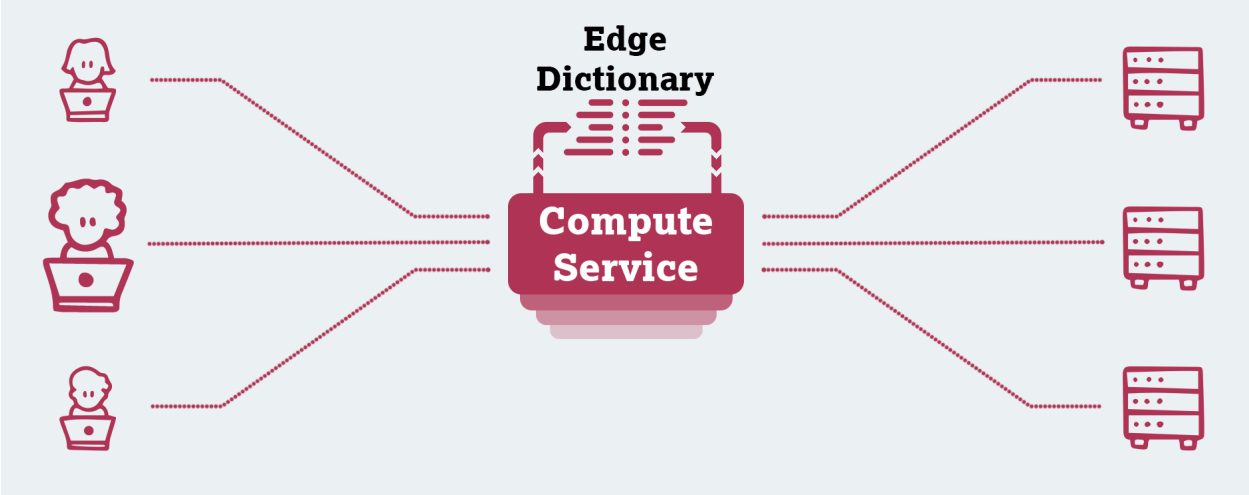


Log Tailing is a feature that improves the developer experience. For a developer, on the left above, debugging an app can be difficult without access to error logs. Live error logs are even better and Log Tailing allows you to receive your app's logs live via the Fastly CLI. This feature can be toggled on/off by the Fastly CLI, allowing you to control when you wish to receive logs or not.

For some developers, running live testing on Fastly's infrastructure is a few extra steps too many and would instead prefer to test locally. The Fastly CLI allows developers to run **Local Testing** by setting up a WasmTime local server on their device. WasmTime is another WebAssembly runtime very similar to Lucet.



The Local Testing server will then run their WASM Binary. Local ports will be announced to allow Developers to send test traffic to one port, and receive live logs via the other port.



Fastly's **Edge Dictionary** feature, originally developed for our Delivery service configurations, uses key-value pair data tables for things like IP lists or URL redirects, to be updated via API calls instead of requiring a version update of the service configuration. These Edge Dictionaries, since they are not tied to a specific version of the service configuration, are referred to as Versionless Edge Dictionaries. Edge Dictionaries are also supported with Compute services.

Dynamic Backends allow developers to make connections from their Fastly Compute@Edge services to origins that haven't been defined in advance as part of the service configuration. This unlocks the ability to send requests to origins that aren't known in advance, and origins that change from time to time. Moreover, dynamic backends allow customers to connect to new origins without deploying a new version of their service. Dynamic backend support, though generally available, requires enablement by Fastly customer support. Interfaces for dynamic backend are included in Fastly's Rust and JavaScript SDKs for Compute@Edge.

Relevant Links:
https://developer.fastly.com/learning/compute/testing/
https://www.fastly.com/blog/dynamic-backends-takes-the-pain-out-of-backend-configuration-management